



CSI247: Data Structures

Lecture 07 – Time complexity of algorithms

Department of Computer Science
B. Gopolang (247-275)

Previous lesson

- Recursion
 - Advantages vs disadvantages
- Implementation of a recursive method
 - Base case - for method termination
 - Recursive case(s)
- Using trace tables to trace recursive methods
- Infinite recursion
- Converting between a recursive and iterative methods

Today ...

Learning Objectives

- By the end of this lecture you must be able to:
 - Determine growth rate of an algorithm.
 - Determine Big-O values for a given algorithm

Introduction

- Rem :
 - Programs are written to solve problems
- A program: Implementation of an *algorithm*
 - Algorithm: A well-defined instruction set to solve a specific problem in a finite amount of time
- Notes:
 - Several possible algorithms for 1 problem
 - Various programming language to implement an algorithm
 - Different implementation platforms available

Efficiency of an algorithm

- A measure of how fast an algorithm runs
 - Affected by its performance & available resources:
 - a) Time it takes (this handout's focus)
 - Instructions take time to execute
 - Qs: - Is our algorithm efficient enough?
 - What issue affect the algorithm's runtime?
 - b) Storage space
 - Data to be processed must be stored somewhere
 - Qs - Which data structure(s) to use?
 - How will it affect the algorithm's efficiency?
 - We can always increase space! What about time?

Time complexity of an algorithm

- **Time complexity!**

- Time taken by an algorithm to run given input of size n
 - Used to estimate algorithm's runtime

- Runtime:

- i. Depends on input data

- E.g.:

- Which array is easier and quicker to search?
 - An ordered (sorted) & an unsorted one

- ii. Increases when input size (n) increases

- More time needed to process large data sets
- Array of size 10 vs array of size 1000?

- Let's say:
 - We have 2 potential algorithms for 1 problem
 - Q: How can we know which one is efficient?
 - A: We need to estimate time taken by each
 - Then compare them!
- NB:
 - Good analysis must be independent from
 - h/w (architecture, clock speed, etc)
 - data set

a) Option 1: Experimentation

- Write programs to implement both algorithms (e.g. in Java)
- Run 2 programs, using identical data sets of different sizes (**n**)
- Record their runtimes, several times
 - Try using `System.currentTimeMillis()` for a measure of the actual execution time
- Compute the average runtimes
- Compare the average runtimes
 - The shorter the runtime, the better!
- Can plot the results in a graph

Limitations

- Must implement the algorithms
- Same environment (h/w and s/w) required for both
- Will we get same results for all data sets?
 - Experiments usually use limited input sets
 - So, algorithm may be less efficient on some data later
- Other factors likely to affect runtime worth considering:
 - Is h/w used representative of all h/w?
 - Some are more powerful than others
 - Programming style
 - Clean code vs spaghetti code

b) Option 2: Methodological approach

- **Forget specific running time**
 - Uses Mathematical techniques to estimate complexity
 - No need for implementation of the algorithm
- Runtime defined as a function of input size, n .
 - Focuses on algorithm's scalability
 - Expressed using some standard notation
- Based on the number of primitive operations in the algorithm (pseudo code or program)

- Primitive operations:

- Algorithm's basic low-level actions, easily identifiable
- Independent of the language & h/w
- Assumption:
 - Each takes some constant time, c , to execute
- E.g.:
 - Declaration statements, arithmetic operations, assignment statements, evaluating expressions, array indexing, method calls, etc.

Possible estimates:

a) Best-case

- Shortest possible runtime
 - Assumes all will always go well ...

b) Average-case

- Expected runtime over input size **n**
 - Difficult to calculate

c) Worst-case

- Gives maximum runtime for an input size **n**
 - Most commonly used
- Most crucial in some application domains (e.g. surgery, air traffic control, etc.)

- How to estimate runtime:
 - Count number of primitive operations
 - Express the efficiency using growth functions
 - How runtime increases when size increases

Notes:

- Convention:
 - Parameterize running time with input size
 - E.g.:
 - $T_a(n)$: estimated time of algorithm **a** given input size **n**

Calculating estimated runtimes

- Each primitive operation takes some time
- Examples

a) 1 statement:

$i = 1;$ $// \mathbf{c_1}$, time: 1 (constant time)

b) Sequential statements:

$i = 1;$ $// \mathbf{c_1}$, time: 1

$val += 1;$ $// \mathbf{c_2}$, time: 1

Total time, $\mathbf{T(n)} = 1 * \mathbf{c_1} + 1 * \mathbf{c_2}$

c) Selection

if (a > o) // c_1 , time: 1

 val = 10; // c_2 , time: 1

else

 val = 1; // c_3 , time: 1

$$\begin{aligned}\text{Total time, } T(n) &= 1 * c_1 + \max(1 * c_2, 1 * c_3) \\ &= 1 + \max(1, 1)\end{aligned}$$

$T(n)$ is constant

d) Basic repetition block

```
tot = 0;           //  $c_1$ , 1
c = 1;            //  $c_2$ , 1
while (c <= n) {   //  $c_3$ , executed  $n+1$  times
    tot = tot + c;  //  $c_4$ , 1 * n times ( $=n$  times)
    c = c + 2;      //  $c_5$ , 1 * n times
}
```

Total time, $T(n) = c_1 + c_2 + (n+1)*c_3 + n*c_4 + n*c_5$

$T(n)$ is proportional to n

e) Nested repetition block

```
c=1;           //  $c_1$ , 1
while (c <= n) { //  $c_2$ ,  $n+1$  times
    i=1;        //  $c_3$ ,  $n$  times
    while (i <= n) { //  $c_4$ ,  $(n+1) * n$  times
        print i;    //  $c_5$ ,  $n * n$  times
        i = i+1;    //  $c_6$ ,  $n * n$  times
    }
    c = c+1;       //  $c_7$ ,  $n$  times
}
```

$$T(n) = c_1 + (n+1) * c_2 + n * c_3 + n * (n+1) * c_4 + \mathbf{n * n * c_5} + \mathbf{n * n * c_6} + n *$$

$$T(n) = \dots?$$

$T(n)$ is proportional to n^2

● Estimation rules

- **Sequential statements:** sum runtimes of individual statements.
- **if-else:** At most, runtime of condition + max between runtimes of if-block and else-block
- **Loops:** runtime of loop statements * no. of iterations

Algorithm Growth Rates

- Assume algorithms have these runtimes for size n :
 - Algorithm A: $2*n$
 - Algorithm B: $4 + 3*n^2$
- From these, we can then derive **growth rates**
 - Refers to increase in runtime as size increases
 - Look for the fastest growing term
 - A: time is proportional to n (linear rate).
 - B: time is proportional to n^2 (quadratic rate).
- Growth rates can help compare efficiency of algorithms

Big-Oh and Growth Rate

- We can say “ $f(n)$ is $O(g(n))$ ” // worse case
 - Means growth rate of $f(n)$ does not exceed growth rate of $g(n)$
- This notation is called the **Big O notation**
- Provides a formal way to say $10n - 2 \approx n$
- We can rank algorithms' growth rate using Big-Oh notation
 - An algorithm whose time is proportional to n^2 , is said to be $O(n^2)$.
 - Algorithm requiring time proportional to n , it is $O(n)$.

Big-Oh Rules

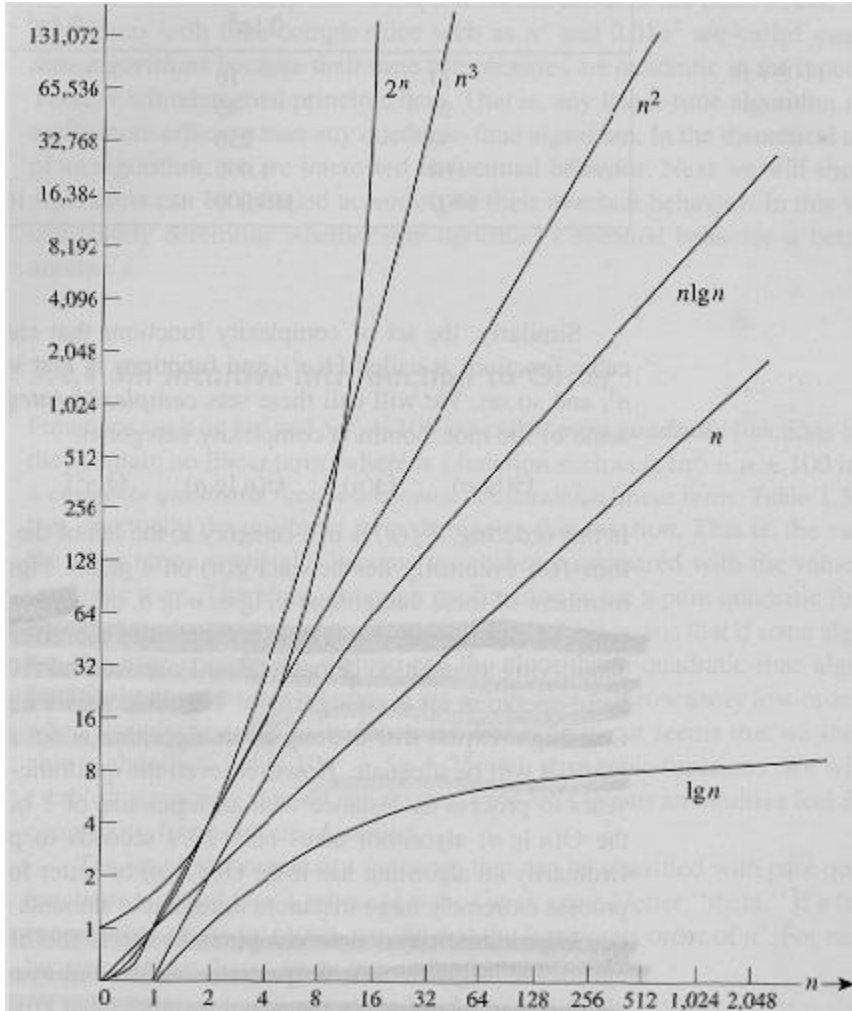
- Focus on the dominant term
 - Drop constants, coefficients & lower-order terms
 - Keep the fastest growing term
 - **Rem: Worst-case performance**
- Why?
 - They do not affect growth rate
 - Increasing n does not alter them!
- Examples (Big-O for some runtimes)

$$f(n) = 7n - 3 \quad \rightarrow \quad O(n)$$

$$f(n) = 8n^2 \log n + 5n^2 + n \quad \rightarrow \quad O(n^2 \log n)$$

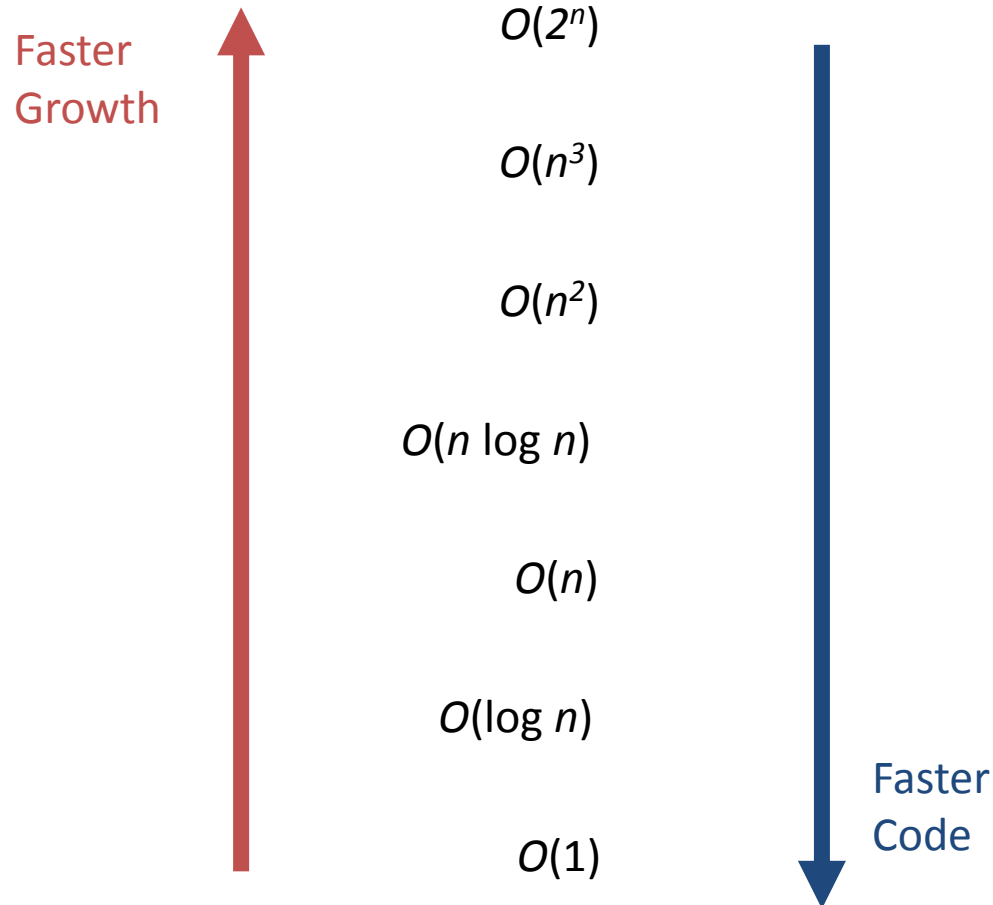
$$f(n) = 3n^2 + 5n \quad \rightarrow \quad O(n^2)$$

Common orders of Big-O magnitude



- Common time complexities:
 - Constant ≈ 1
 - Logarithmic $\approx \log n$
 - Linear $\approx n$
 - N-Log-N $\approx n \log n$
 - Quadratic $\approx n^2$
 - Cubic $\approx n^3$
 - Exponential $\approx 2^n$

Comparing Growth Rates



Exercises

- Determine Big-Oh for each of these growth rates:
 - $7n - 2$
 - $3n^3 + 20n^2 + 5$
 - $3 \log n + 5$
- Derive time complexity, using big-O notation, of these:
 - An algorithm that determines the average of n numbers
 - An algorithm that prints all characters in a string of length n
 - An iterative algorithm for computing $n!$
 - Note: derive the algorithms first

Summary

- In this lesson, we looked at how to:
 - Determine growth rate of an algorithm
 - Determine Big-O values for a given algorithm

Next lesson

- Sorting arrays !!!

Q & A



Thank you